One standard error rule, regularization

Fraida Fund

Contents

One standard error rule . . . 2
- One-SE rule . . . 2
- One-SE rule: compute standard error . . . 2
- One-SE rule: Step 1 with lower-is-better metric (formula) . . . 2
- One-SE rule: Step 1 with lower-is-better metric (code) . . . 3
- One-SE rule: Step 1 with higher-is-better metric (formula) . . . 3
- One-SE rule: Step 1 with higher-is-better metric (code) . . . 3
- One-SE rule: Step 2 with models ordered from least complex to most (formula) . . . 4
- One-SE rule: Step 2 with models ordered from least complex to most (code) . . . 4
- One-SE rule: Step 2 with models ordered from most complex to least (formula) . . . 4
- One-SE rule: Step 2 with models ordered from most complex to least (code) . . . 4

Regularization . . . 5
- Penalty for model complexity . . . 5
- Regularization vs. standard LS . . . 5
- Common regularizers . . . 5
- Graphical representation . . . 6
- Standardization . . . 7
- Ridge with standardization . . . 7
- LASSO with standardization . . . 7
- Ridge term and derivative . . . 8
- LASSO term and derivative . . . 8
- Effect of regularization level . . . 8
- Selecting regularization level . . . 8

One standard error rule

- Model selection that minimizes mean error often results in too-complex model
- One standard error rule: use simplest model where mean error is within one SE of the minimum mean error

One-SE rule

- **Step 1**: Find models within one SE of "best" model
- **Step 2**: Pick the simplest model from the list of candidates

One-SE rule: compute standard error

- Given data X, y
- Compute score $S_{p,i}$ for model p on fold i (of K)
- Compute average ($\bar{S}_p$), standard deviation σ_p, and standard error of scores:

$$SE_p = \frac{\sigma_p}{\sqrt{K-1}}$$

One-SE rule: Step 1 with lower-is-better metric (formula)

"Best score" model selection: $p^* = \text{argmin}_p \bar{S}_p$

One SE rule for "lower is better" scoring metric: Compute target score: $S_t = \bar{S}_{p^*} + SE_{p^*}$

then candidate models are those with score lower than target:

$$\mathcal{P}^{\text{1SE}} = \{p | \bar{S}_p \leq S_t\}$$

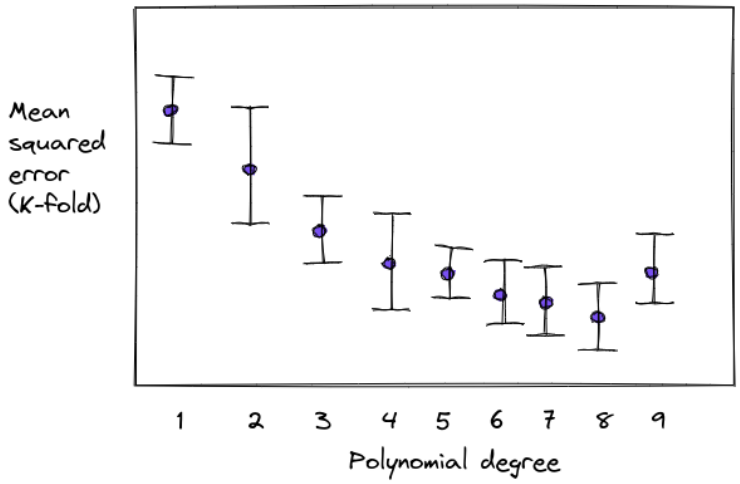


Figure 1: Model selection using one SE rule on MSE. The best scoring model is $d = 8$, but $d = 6$ is simplest model within one SE of the best scoring model, and so $d = 6$ would be selected according to the one-SE rule.

Note: this assumes you are using a "smaller is better" metric such as MSE. If you are using a "larger is better" metric, like R2, how would we change the algorithm?

One-SE rule: Step 1 with lower-is-better metric (code)

```
# score_val is a lower-is-better metric
# so threshold is: SMALLEST mean score PLUS its std/(nfold-1)
idx_min = np.argmin(score_val.mean(axis=1))
target = score_val[idx_min,:].mean() + score_val[idx_min,:].std()/np.sqrt(nfold-1)
# candidate models have mean score BELOW that threshold
idx_one_se = np.where(score_val.mean(axis=1) <= target)
```

One-SE rule: Step 1 with higher-is-better metric (formula)

"Best score" model selection: $p^* = \text{argmax}_p \bar{S}_p$

One SE rule for "higher is better" scoring metric: Compute target score: $S_t = \bar{S}_{p^*} - SE_{p^*}$

then candidate models are those with score higher than target:

$$\mathcal{P}^{\text{1SE}} = \{p | \bar{S}_p \geq S_t\}$$

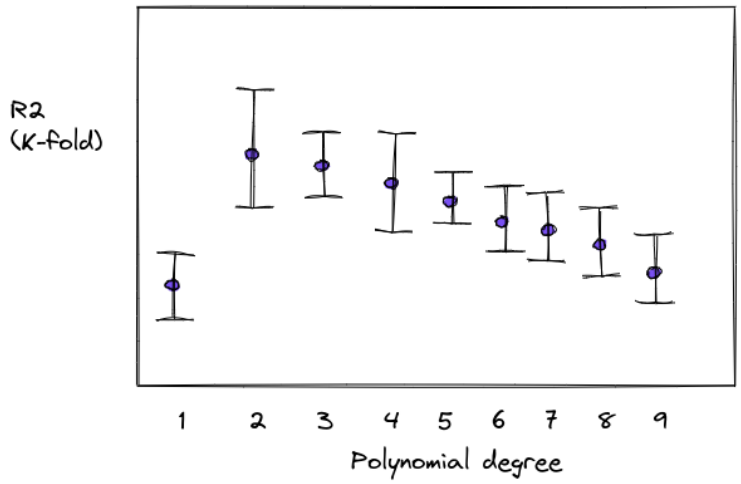


Figure 2: Model selection using one SE rule on R2. In this example, the best scoring model is $d = 2$, and there is no simpler model within one SE, so the one-SE rule would also select $d = 2$.

One-SE rule: Step 1 with higher-is-better metric (code)

```
# score_val is a higher-is-better metric
# so threshold is: LARGEST mean score MINUS its std/(nfold-1)
idx_min = np.argmax(score_val.mean(axis=1))
target = score_val[idx_min,:].mean() - score_val[idx_min,:].std()/np.sqrt(nfold-1)
# candidate models have mean score ABOVE that threshold
idx_one_se = np.where(score_val.mean(axis=1) >= target)
```

One-SE rule: Step 2 with models ordered from least complex to most (formula)

Select simplest model from the candidates, i.e. the one with the lowest index:

$$p^{*,1\text{SE}} = \min \mathcal{P}^{1\text{SE}}$$

One-SE rule: Step 2 with models ordered from least complex to most (code)

```
# get LOWEST indexed model (least complex)
m_one_se = np.min(model_list[idx_one_se])
```

One-SE rule: Step 2 with models ordered from most complex to least (formula)

Select simplest model from the candidates, i.e. the one with the highest index:

$$p^{*,1\text{SE}} = \max \mathcal{P}^{1\text{SE}}$$

One-SE rule: Step 2 with models ordered from most complex to least (code)

```
# get HIGHEST indexed model (least complex)
m_one_se = np.max(model_list[idx_one_se])
```

Regularization

Penalty for model complexity

With no bounds on complexity of model, we can always get a model with zero training error on finite training set - overfitting.

Basic idea: apply penalty in loss function to discourage more complex models

This is an alternative way to manage the bias-variance tradeoff. The regularization "level" controls the "flexibility" of the model, and therefore, its position on the bias/variance curve.

Regularization vs. standard LS

Least squares estimate:

$$\widehat{w} = \underset{w}{\text{argmin}}\, MSE(w), \quad MSE(w) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \widehat{y}_i)^2$$

Regularized estimate w/ **regularizing function** $\phi(w)$:

$$\widehat{w} = \underset{w}{\text{argmin}}\, J(w), \quad J(w) = MSE(w) + \phi(w)$$

Common regularizers

Ridge regression (L2):

$$\phi(w) = \alpha \sum_{j=1}^{d} |w_j|^2$$

LASSO regression (L1):

$$\phi(w) = \alpha \sum_{j=1}^{d} |w_j|$$

Graphical representation

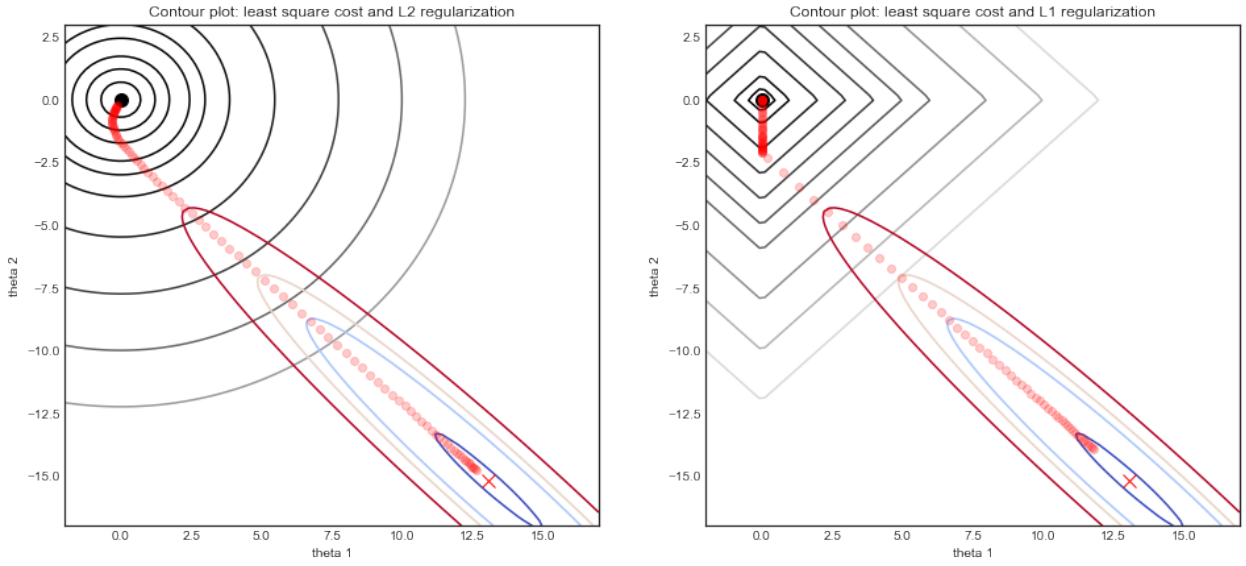


Figure 3: LS solution (+), RSS contours. As we increase α, regularized solution moves from LS to 0.

Common features: Ridge and LASSO

- Both penalize large w_j
- Both have hyperparameter α that controls level of regularization
- Intercept w_0 not included in regularization sum (starts at 1!), this depends on mean of y and should not be constrained.

Specific to Ridge (L2):

- minimizes $|w_j|^2$,
- minimal penalty for small non-zero coefficients
- heavily penalizes large coefficients
- tends to make many "small" coefficients
- Not for feature selection

Specific to LASSO (L1):

- minimizes $|w_j|$
- tends to make coefficients either 0 or large (sparse!)
- does feature selection (setting w_j to zero is equivalent to un-selecting feature)

To understand why L1 regularization tends to make sparse coefficients but not L2 regularization - look at the graphical representation. Note that the contours of the L1 regularization "stick out" when one or both parameters is zero.

Standardization

Before learning a model with regularization, we typically *standardize* each feature and target to have zero mean, unit variance:

- $x_{i,j} \rightarrow \frac{x_{i,j} - \bar{x}_j}{s_{x_j}}$
- $y_i \rightarrow \frac{y_i - \bar{y}}{s_y}$

Why?

- Without scaling, regularization depends on data range
- With mean removal, no longer need w_0, so regularization term is just L1 or L2 norm of coefficient vector

Important note:

- Use mean, variance of *training data* to transform training data
- **Also** use mean, variance of *training data* to transform **test data**

Ridge with standardization

Assuming data standardized to zero mean, unit variance, the Ridge cost function is:

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{d} |w_j|^2 \\ &= ||\mathbf{A}\mathbf{w} - \mathbf{y}||^2 + \alpha ||\mathbf{w}||^2 \end{aligned}$$

LASSO with standardization

LASSO cost function ($||\mathbf{w}||_1$ is L1 norm):

$$\begin{aligned} J(\mathbf{w}) &= \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^{d} |w_j| \\ &= ||\mathbf{A}\mathbf{w} - \mathbf{y}||^2 + \alpha ||\mathbf{w}||_1 \end{aligned}$$

Ridge term and derivative

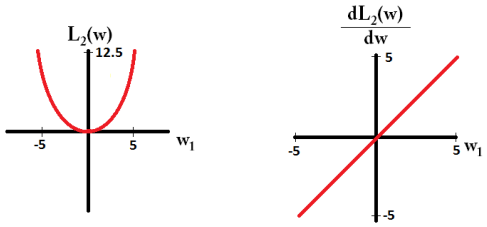


Figure 4: L2 term and its derivative for one parameter.

$$J(\mathbf{w}) = ||\mathbf{A}\mathbf{w} - \mathbf{y}||^2 + \alpha||\mathbf{w}||^2$$

Taking derivative:

$$\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}} = -2\mathbf{A}^T(\mathbf{y} - \mathbf{A}\mathbf{w}) + 2\alpha\mathbf{w}$$

Setting it to zero, we find

$$\mathbf{w}_{\text{ridge}} = (\mathbf{A}^T\mathbf{A} + \alpha\mathbf{I})^{-1}\mathbf{A}^T\mathbf{y}$$

LASSO term and derivative

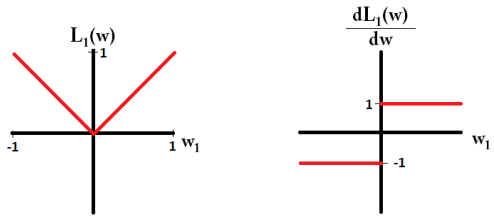


Figure 5: L1 term and its derivative for one parameter.

- No closed-form solution: derivative of $|w_j|$ is not continuous
- But there is a unique minimum, because cost function is convex, can solve iteratively

Effect of regularization level

Greater α, less complex model, more bias!

- Ridge: Greater α makes coefficients smaller.
- LASSO: Greater α makes more weights zero.

Selecting regularization level

How to select α? by CV!

- Outer loop: loop over CV folds
- Inner loop: loop over α